

Fiche de révision ComptaFlow

Comprendre l'architecture, les flux et le rôle de chaque fichier



RADOUI Fatima Zahra

Génie informatique — ENSA de Berrechid

Version fondée sur le code réel du dépôt — août 2026

Table des matières

1	Comment utiliser cette fiche	1
1.1	Périmètre	1
1.2	Trois questions à poser pour chaque fichier	1
2	Vue d'ensemble de l'architecture	2
2.1	Architecture d'exécution	2
2.2	Règle d'isolation multi-tenant	2
3	Flux fonctionnels à connaître	3
3.1	Connexion et autorisation	3
3.2	Import et traitement d'un document	4
3.3	Facture vers pré-écriture et lignes comptables	5
3.4	Rapprochement bancaire	6
3.5	Du frontend à l'écran	6
4	Backend : rôle de chaque fichier	7
4.1	Point d'entrée et cœur technique	7
4.2	Modèles SQLAlchemy	7
4.3	Schémas Pydantic	9
4.4	Routes FastAPI	10
4.5	Services métier	11
4.5.1	Services documentaires, OCR et IA	11
4.5.2	Services comptables	12
4.5.3	Services banque, devises et organisation	13
4.6	Tâches, utilitaires et scripts	13
4.7	Alembic et évolution de la base	14
4.8	Tests backend	15
5	Frontend : rôle de chaque fichier	17
5.1	Entrée, contexte et composants	17
5.2	Client HTTP et modules API	17
5.3	Types TypeScript	18
5.4	Pages React	19
5.5	Utilitaires, styles et ressources	20

6	Configuration, déploiement et documentation	21
7	Carte de dépendances pour réviser rapidement	23
7.1	Questions de soutenance et réponses courtes	23
7.2	Ordre conseillé pour relire le code	24

Chapitre 1

Comment utiliser cette fiche

Cette fiche sert à préparer une soutenance, expliquer le projet à un enseignant et retrouver rapidement le chemin d'une fonctionnalité dans le code. Elle ne se contente pas de reprendre les noms des fichiers : elle indique qui appelle qui, quelles données circulent et où sont appliquées les règles importantes.

1.1 Périmètre

Sont décrits individuellement : les fichiers applicatifs du backend et du frontend, les modèles, schémas, routes, services, tâches Celery, migrations Alembic, scripts, tests, fichiers Docker et fichiers de configuration. Sont exclus de l'inventaire détaillé : `node_modules/`, `__pycache__/`, `.git/`, les fichiers importés dans `storage_local/`, les captures, les résultats JSON de benchmark et les fichiers temporaires produits par LaTeX. Ces éléments sont des dépendances, des données ou des artefacts, pas du code à apprendre.

1.2 Trois questions à poser pour chaque fichier

1. **Qui l'appelle ?** Une route, une page React, une tâche Celery, Alembic ou un test.
2. **Que transforme-t-il ?** Une requête HTTP, un document, un objet SQLAlchemy, des lignes comptables ou un état React.
3. **Que renvoie-t-il ou persiste-t-il ?** Un schéma Pydantic, une réponse JSON, un fichier exporté ou une modification PostgreSQL.

Couche	Idee à retenir
Page React	Affiche l'état, collecte les actions et appelle un fichier de <code>frontend/src/api/</code> .
API TypeScript	Traduit une action d'interface en requête HTTP Axios typée.
Route FastAPI	Authentifie, contrôle le rôle et le périmètre, valide l'entrée puis appelle les services.
Service métier	Contient les règles comptables, documentaires ou bancaires réutilisables.
Modèle SQLAlchemy	Décrit la table persistée, ses clés étrangères et ses contraintes.
Schéma Pydantic	Définit le contrat JSON d'entrée ou de sortie de l'API.
Migration Alembic	Fait évoluer physiquement PostgreSQL de manière versionnée.
Test	Prouve une règle isolée et protège contre une régression.

Chapitre 2

Vue d'ensemble de l'architecture

2.1 Architecture d'exécution

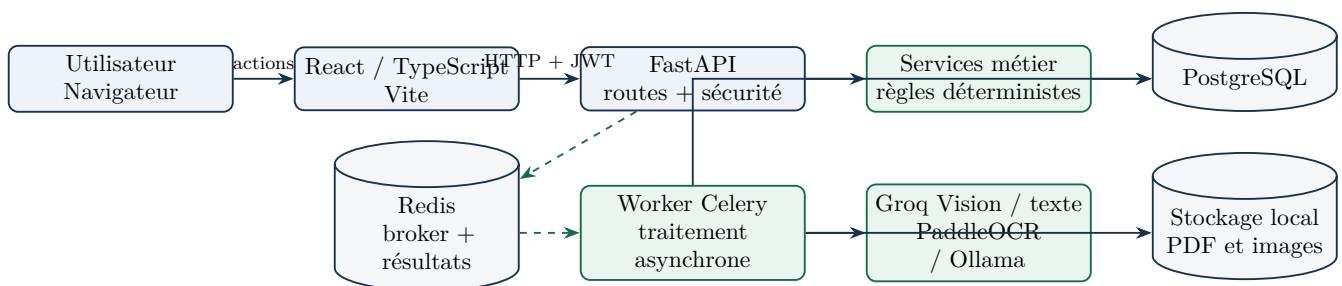
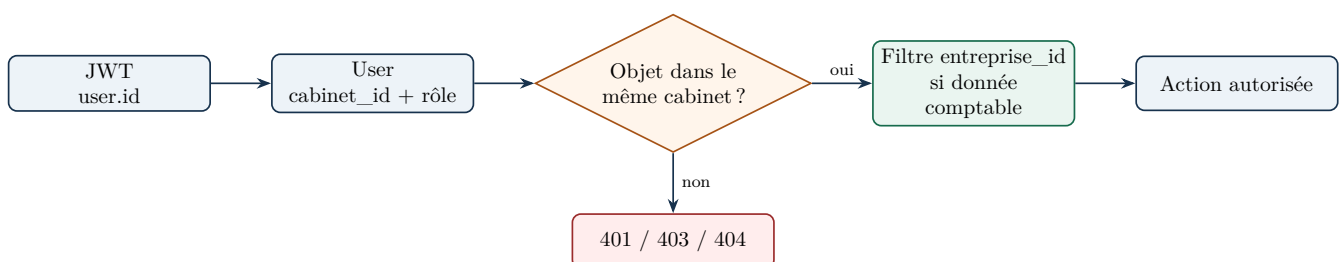


FIGURE 2.1 – Composants actifs et circulation principale

Lecture du schéma. Le navigateur ne parle jamais directement à PostgreSQL. Une page appelle Axios ; Axios ajoute le JWT ; FastAPI vérifie l'identité et le rôle. La route filtre d'abord par `cabinet_id`, puis par `entreprise_id` lorsque la donnée appartient à un dossier comptable. Pour un traitement lourd, FastAPI dépose seulement une tâche dans Redis et rend la main. Celery reprend le travail et écrit progressivement les résultats en base.

2.2 Règle d'isolation multi-tenant

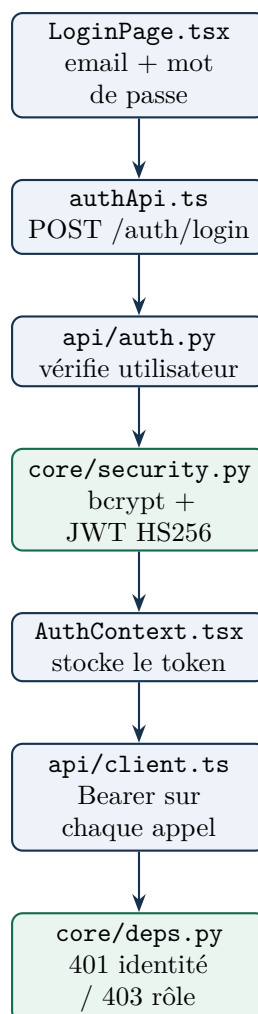


Le `cabinet_id` vient de l'utilisateur authentifié, jamais d'un cabinet arbitraire envoyé par le navigateur. Pour les comptes, écritures, mouvements, rapprochements et états, le filtre `entreprise_id` complète ce premier périmètre.

Chapitre 3

Flux fonctionnels à connaître

3.1 Connexion et autorisation



Le mot de passe n'est jamais stocké en clair. Après connexion, le frontend conserve le JWT dans `localStorage`. L'intercepteur Axios l'ajoute au header `Authorization`. Si le backend répond 401, l'intercepteur supprime le token et redirige vers la page de connexion.

3.2 Import et traitement d'un document

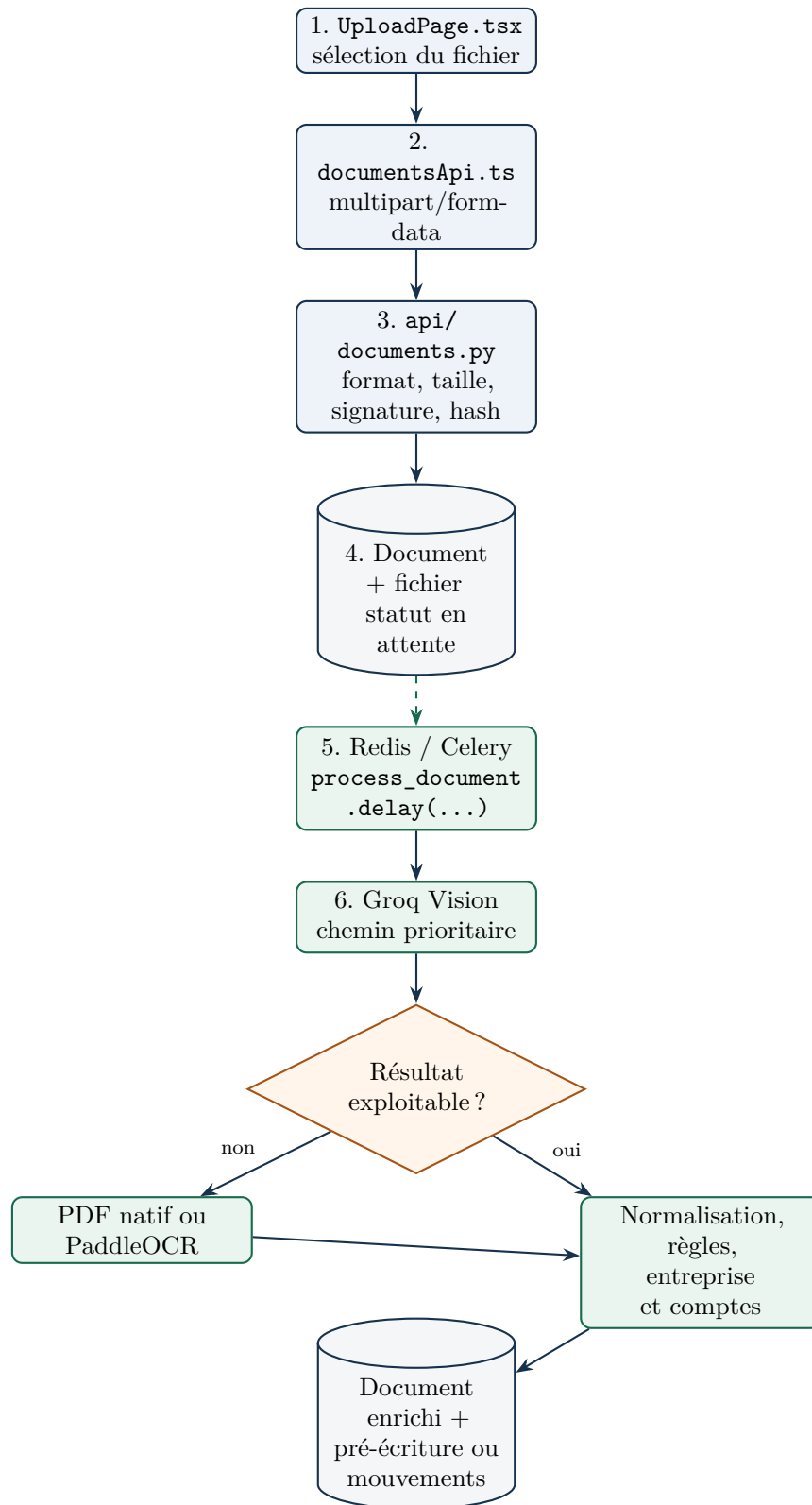


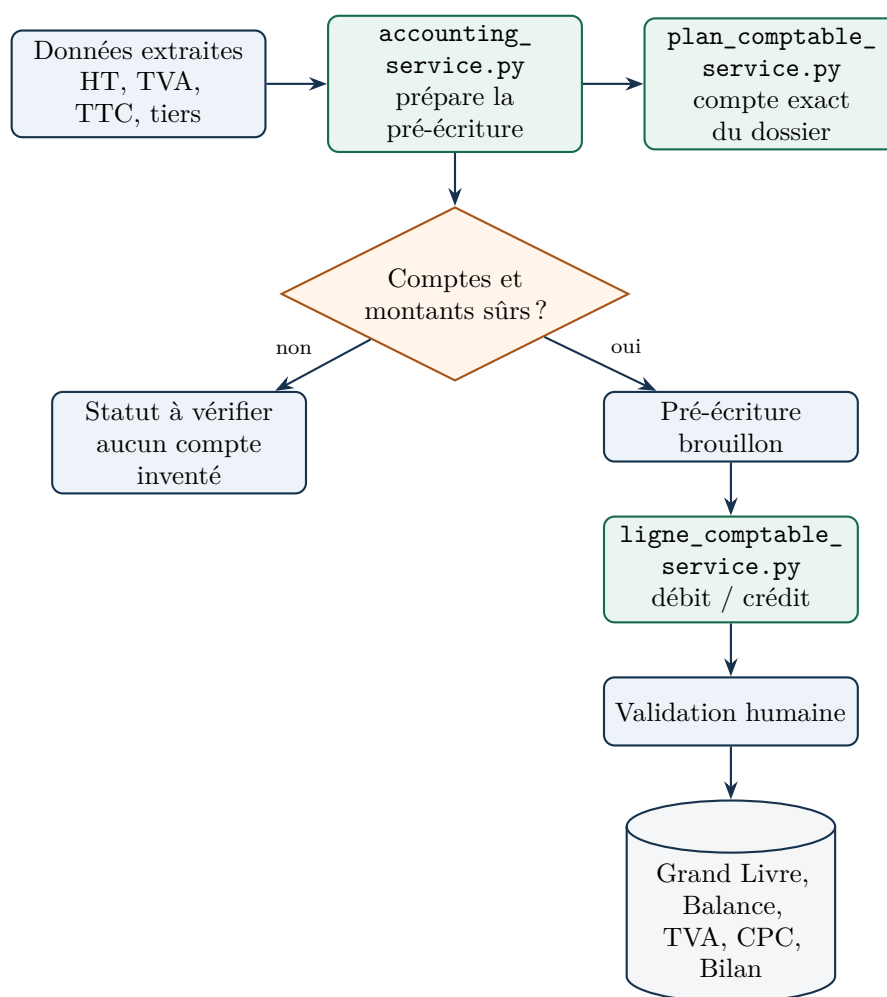
FIGURE 3.1 – Chaîne documentaire simplifiée

Étapes clés dans `tasks/document_processing.py`.

1. ouvrir une session SQLAlchemy indépendante du cycle HTTP ;
2. tenter l'extraction Vision sur les pages préparées ;

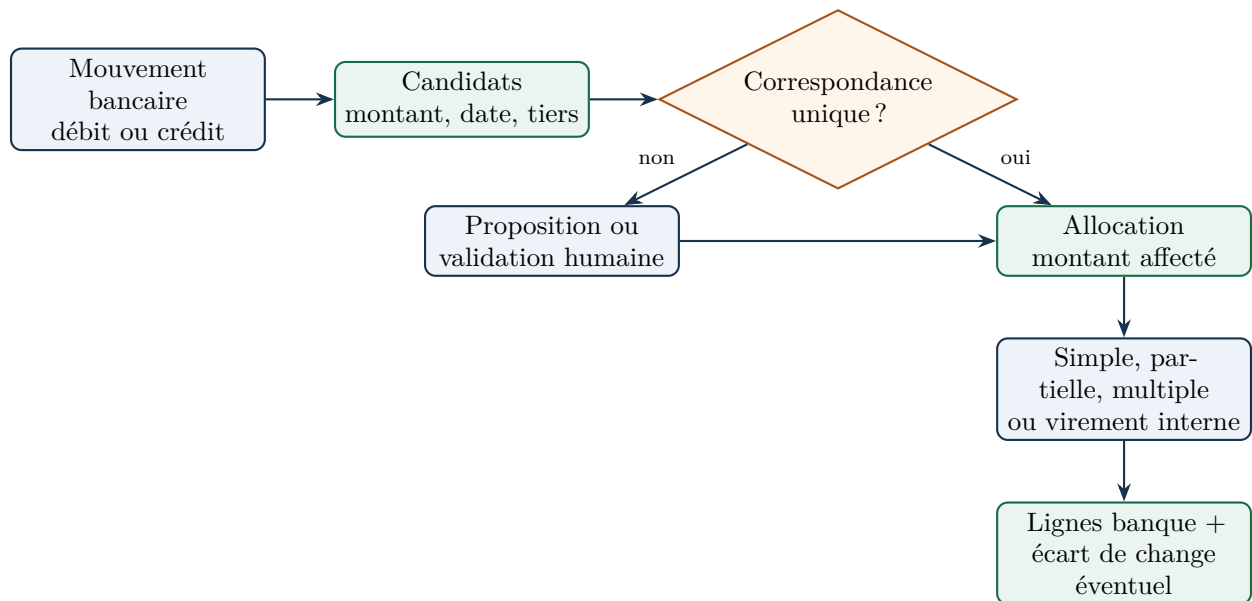
3. en repli, lire le texte natif du PDF ou exécuter PaddleOCR ;
4. reconnaître un relevé bancaire ou une pièce classique ;
5. compléter sans écraser silencieusement les valeurs extraites ;
6. identifier l'entreprise gérée et le sens achat/vente ;
7. classer le document dans le chrono principal et sa période ;
8. créer ou mettre à jour une pré-écriture, ou recréer les mouvements bancaires ;
9. enregistrer le statut final, les erreurs structurées et les métriques utiles.

3.3 Facture vers pré-écriture et lignes comptables



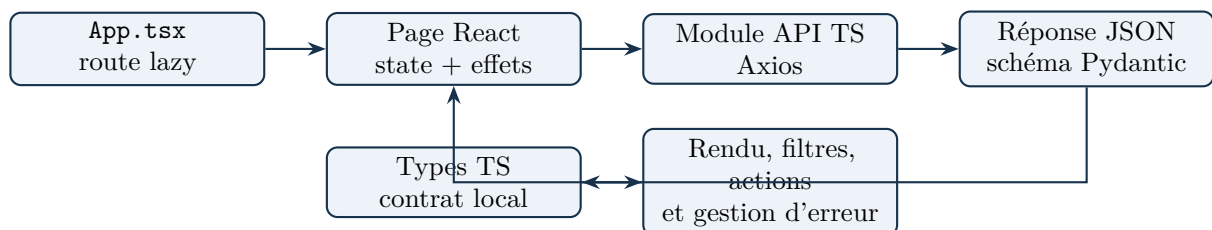
L'IA peut proposer une *nature économique* ; elle ne choisit pas arbitrairement un numéro complet. Python traduit la nature en famille CGNC, puis le service du plan comptable cherche un compte actif propre à l'entreprise. Zéro ou plusieurs candidats sûrs entraînent une vérification humaine. Seules les lignes marquées validées alimentent les états.

3.4 Rapprochement bancaire



La table d'allocation porte le montant affecté : cela permet plusieurs paiements pour une facture et plusieurs factures pour un paiement. Le service interdit d'affecter plus que le solde restant. Les virements internes lient deux mouvements opposés sans les compter deux fois.

3.5 Du frontend à l'écran



Chapitre 4

Backend : rôle de chaque fichier

4.1 Point d'entrée et cœur technique

TABLE 4.1: Point d'entrée et fichiers backend/app/core/

Fichier	Rôle et fonctionnement	État
backend/app/__init__.py	Marque app comme package Python. Il ne porte pas de règle métier.	Support
backend/app/main.py	Crée l'application FastAPI, configure CORS, branche tous les routers, ajoute un identifiant de requête et mesure la durée HTTP. Expose aussi /health , /ready et la racine.	Actif
backend/app/core/__init__.py	Marque le répertoire core comme package.	Support
backend/app/core/config.py	Charge les paramètres depuis l'environnement : base, secret JWT, Redis, stockage, Ollama, Groq et CORS. La propriété cors_origins transforme la chaîne en liste.	Actif
backend/app/core/database.py	Crée l'engine SQLAlchemy, SessionLocal , la classe Base et la dépendance get_db qui ouvre puis ferme une session par requête.	Actif
backend/app/core/security.py	Centralise bcrypt et les JWT HS256 de huit heures : hachage, vérification, création et décodage.	Actif
backend/app/core/deps.py	Résout l'utilisateur depuis le Bearer token, refuse les comptes inactifs et fabrique les contrôles RBAC avec require_role .	Actif
backend/app/core/celery_app.py	Configure Celery avec Redis comme broker et backend, accusé tardif, prélecture 1 et préchauffage Ollama au démarrage du worker.	Actif
backend/app/core/exceptions.py	Définit la hiérarchie d'erreurs du pipeline : définitives, transitoires, fichier illisible et OCR vide. Elle permet de décider si Celery doit réessayer.	Actif
backend/app/core/logging_config.py	Configure les logs de l'API et du worker et injecte le request_id dans chaque ligne pour suivre une opération.	Actif

4.2 Modèles SQLAlchemy

Un modèle décrit ce qui est réellement persisté. Les mixins de **base.py** ajoutent UUID, dates et, pour la majorité des objets métier, **cabinet_id**.

TABLE 4.2: Fichiers backend/app/models/

Fichier	Rôle et fonctionnement	État
models/__init__.py	Importe les modèles pour construire une métadonnée SQLAlchemy complète, notamment pour Alembic.	Support
models/base.py	Fournit UUIDMixin, TimestampMixin et CabinetScopedMixin. Les modèles héritent ainsi des mêmes colonnes techniques.	Actif
models/enums.py	Source commune des valeurs contrôlées : rôles, catégories et statuts documentaires, types d'écriture, TVA, tâches et mouvements.	Actif
models/cabinet.py	Tenant principal : identité du cabinet, état actif, relations vers utilisateurs et entreprises.	Actif
models/user.py	Utilisateur authentifiable : email unique, hash du mot de passe, identité, rôle, activité et dernier accès ; appartient à un cabinet.	Actif
models/entreprise.py	Dossier comptable suivi : nom, ICE, IF, RC, forme, adresse et indicateur de création automatique.	Actif
models/chrono.py	Conteneur documentaire principal d'une entreprise. La période et la catégorie sont portées par les documents, pas par plusieurs chronos physiques.	Actif
models/document.py	Métadonnées du fichier, hash SHA-256, stockage, classement, statut, OCR, JSON extrait, erreurs et relations vers pré-écritures et mouvements. Entreprise et chrono peuvent être nuls.	Actif
models/ecriture.py	Objet de pré-écriture : type, pièce, tiers, comptes proposés, montants, devise, statut de validation, anomalies et suivi de saisie. Le nom physique de table reste <code>ecritures_comptables</code> .	Actif
models/ligne_comptable.py	Lignes débit/crédit issues d'une facture, d'un mouvement ou d'une régularisation ; portent journal, compte, origine, ordre et validation.	Actif
models/compte_comptable_entreprise.py	Plan comptable propre au dossier : numéro, famille CGNC, usage, nature, tiers, source et activation. L'unicité est scoppée par entreprise.	Actif
models/compte_bancaire_entreprise.py	Comptes bancaires configurés avec banque, RIB/IBAN/BIC, devise et compte comptable exact.	Actif
models/mouvement_bancaire.py	Une opération extraite d'un relevé : sens, montant, solde, devise, compte bancaire, contrepartie, rapprochement et éventuel virement lié.	Actif
models/rapprochement_bancaire_allocation.py	Association plusieurs-à-plusieurs entre mouvement et pré-écriture ; porte montant affecté, score, confirmation et éventuel écart de change.	Actif
models/taux_change_bam.py	Cache partagé des cours BAM par date et devise, avec unité de cotation, cours achat/vente et URL source.	Actif
models/tva_periode.py	Contient quatre modèles : configuration TVA du dossier, période calculée, régularisation et utilisation contrôlée d'un crédit antérieur.	Actif
models/regularisation_cloture.py	Opération de clôture : exercice, nature, montant, comptes, statut, anomalies, extourne et traçabilité de validation/génération.	Actif

Fichier	Rôle et fonctionnement	État
models/tache.py	Tâche datée, éventuellement rattachée à une entreprise, avec créateur, responsable, statut, priorité et récurrence.	Actif
models/audit_log.py	Journal d'actions sensibles : utilisateur, action, détails avant/après et adresse IP, isolés par cabinet.	Actif

4.3 Schémas Pydantic

Les schémas sont les contrats JSON. Ils ne créent pas de tables. **Create/Update** valident une entrée ; **Out** contrôle ce que l'API expose.

TABLE 4.3: Fichiers backend/app/schemas/

Fichier	Rôle et fonctionnement	État
schemas/__init__.py	Déclare le package de schémas.	Support
schemas/auth.py	Réponse de connexion contenant le jeton et son type.	Actif
schemas/user.py	Création, modification, connexion et lecture d'un utilisateur.	Actif
schemas/cabinet.py	Lecture et mise à jour des informations du cabinet.	Actif
schemas/entreprise.py	Entreprise standard et entreprise disponible dans un module donné.	Actif
schemas/chrono.py	Vue aplatie d'un document dans les chronos numériques.	Actif
schemas/document.py	Résumé de document pour import et liste.	Actif
schemas/document_detail.py	Détail combinant document, données extraites, résumé de pré-écriture et mouvements.	Actif
schemas/ecriture.py	Sortie et correction autorisée d'une pré-écriture comptable.	Actif
schemas/mouvement_bancaire.py	Mouvements, filtres, candidats, allocations, mises à jour et virements internes.	Actif
schemas/compte_bancaire.py	Création, correction et lecture d'un compte bancaire d'entreprise.	Actif
schemas/plan_comptable.py	Compte comptable, import en masse et bilan de l'import.	Actif
schemas/ledger.py	Lignes et comptes du Grand Livre, Balance et résultat de reconstruction.	Actif
schemas/registre.py	Registres, options de filtre et synthèses TVA historiques.	Actif
schemas/tva.py	Configuration, périodes, régularisations et vue annuelle TVA V2.	Actif
schemas/cpc.py	Détails du CPC, comparaison N/N-1 et contrôles de cohérence.	Actif
schemas/bilan.py	Rubriques du Bilan, détails par compte et contrôle actif/passif.	Actif
schemas/controls.py	Anomalies, résumés par module, score et statut de pré-clôture.	Actif
schemas/cloture.py	Cycle des régularisations de clôture : création, validation, génération, annulation et synthèse.	Actif
schemas/dashboard.py	Indicateurs de documents, pré-écritures, TVA et activité globale.	Actif
schemas/notification.py	Liste des notifications et statistiques par type/entreprise.	Actif

Fichier	Rôle et fonctionnement	État
schemas/rappel.py	Pré-écritures non saisies et entreprises en retard.	Actif
schemas/tache.py	Création, mise à jour et lecture d'une tâche.	Actif
schemas/rapport.py	Synthèse mensuelle ou annuelle utilisée par la page Rapports et le PDF.	Actif
schemas/search.py	Résultat unifié de recherche avec type, titre, sous-titre et route frontend.	Actif
schemas/system.py	Informations techniques non secrètes sur l'instance et ses capacités.	Actif

4.4 Routes FastAPI

Une route doit rester mince : dépendances, paramètres, portée multi-tenant, appel de service et réponse. Les grandes règles ne doivent pas être dupliquées ici.

TABLE 4.4: Fichiers backend/app/api/

Fichier	Rôle et fonctionnement	État
api/__init__.py	Déclare le package des routers.	Support
api/auth.py	POST /auth/login : vérifie email, activité et mot de passe, actualise le dernier accès et retourne le JWT.	Actif
api/users.py	Profil courant, liste, création et désactivation d'utilisateurs avec restrictions d'administration.	Actif
api/cabinet.py	Lecture et mise à jour du cabinet courant, sans accepter un autre tenant.	Actif
api/entreprises.py	Liste des entreprises du cabinet et sélection des entreprises possédant des données pour un module.	Actif
api/documents.py	Import sécurisé, liste, détail, fichier original, correction, validation/rejet, export unitaire, retraitement et suppression. Gère aussi le stockage et la suppression cohérente des données associées.	Actif
api/chronos.py	Recherche documentaire filtrée par entreprise, année, mois et catégorie ; fournit aussi les années disponibles.	Actif
api/accounting.py	Router central des pré-écritures, banque, comptes bancaires, rapprochements, Grand Livre, Balance, registres, TVA, CPC, Bilan et pré-clôture.	Actif
api/plan_comptable.py	CRUD et import du plan propre à chaque entreprise ; contrôle systématiquement le cabinet et le dossier.	Actif
api/exchange_rates.py	Prévisualise le cours BAM applicable sans modifier une écriture.	Actif
api/tva.py	Recalcule les périodes, configure le dossier, ajoute les régularisations et valide une période TVA.	Actif
api/cloture.py	CRUD et cycle de vie des régularisations de clôture, puis synthèse annuelle.	Actif
api/dashboard.py	Agrège des compteurs scoppés pour la période demandée.	Actif
api/search.py	Recherche transversale dans entreprises, documents et pré-écritures du cabinet.	Actif
api/notifications.py	Construit dynamiquement les alertes et leurs statistiques ; il ne s'agit pas d'une boîte de messages persistée.	Actif

Fichier	Rôle et fonctionnement	État
api/rappels.py	Expose les pré-écritures non saisies et les retards d'entreprise.	Actif
api/taches.py	Liste, création, correction, achèvement avec récurrence et suppression des tâches.	Actif
api/rapports.py	Génère une synthèse JSON ou son PDF pour une entreprise et une période.	Actif
api/export.py	Exporte les registres validés en CSV/XLSX/PDF. L'endpoint Topaze reste volontairement 501, donc aucune intégration directe n'est annoncée.	Actif
api/system.py	Retourne des informations d'environnement autorisées, sans dévoiler les secrets.	Actif

4.5 Services métier

4.5.1 Services documentaires, OCR et IA

TABLE 4.5: Services d'extraction et de classement

Fichier	Rôle et fonctionnement	État
services/__init__.py	Déclare le package des services.	Support
services/preprocessing_service.py	Extrait le texte natif d'un PDF ou transforme ses pages en images nettoyées et redimensionnées ; supprime ensuite les temporaires.	Actif
services/vision_service.py	Chemin prioritaire Groq Vision : encode les images, appelle le modèle avec retries, parse le JSON, normalise catégories/natures et conserve les incohérences de montants.	Actif
services/ocr_service.py	Charge PaddleOCR paresseusement dans le worker, sérialise l'accès par verrou et limite chaque page par un timeout.	Actif
services/groq_service.py	Analyse textuelle Groq utilisée après extraction du texte ; parse et contrôle la réponse, sans fabriquer une TVA absente.	Actif
services/ai_service.py	Repli local : extraction rapide, appel Ollama, nettoyage JSON et préchauffage du modèle. La présence de la clé Groq oriente le chemin principal.	Actif
services/regex_extraction_service.py	Complète les champs manquants par expressions régulières et vérifie HT + TVA = TTC sans remplacer silencieusement les sources.	Actif
services/extraction_mapping_service.py	Transforme les champs hétérogènes de l'extraction en dictionnaire métier stable attendu par les étapes suivantes.	Actif
services/document_classifier.py	Classification heuristique par expressions et scores : facture, banque et autres catégories documentaires.	Actif
services/ml_classifier_service.py	Charge, si disponible, un classifieur scikit-learn entraîné et retourne catégorie + confiance ; le pipeline peut continuer sans lui.	Actif

Fichier	Rôle et fonctionnement	État
<code>services/bank_statement_service.py</code>	Extrait une page ou un texte bancaire, conserve les lignes répétées, normalise dates/montants et vérifie la cohérence des soldes.	Actif
<code>services/company_service.py</code>	Distingue entreprise gérée et tiers en comparant ICE/IF/RC et noms normalisés ; détermine achat/vente selon émetteur/destinataire.	Actif
<code>services/chrono_service.py</code>	Obtient ou crée le chrono principal du dossier, sous portée cabinet + entreprise.	Actif
<code>services/anomaly_service.py</code>	Détecte les doublons métier, relie un doublon potentiel et conserve les motifs au lieu de supprimer une opération silencieusement.	Actif

4.5.2 Services comptables

TABLE 4.6: Services de préparation et d'états comptables

Fichier	Rôle et fonctionnement	État
<code>services/accounting_rules_service.py</code>	Tables de correspondance entre nature économique et familles CGNC pour achats, ventes et TVA ; ne produit pas un compte exact du dossier.	Actif
<code>services/plan_comptable_service.py</code>	Normalise et valide un compte, importe le plan et résout un compte exact uniquement lorsqu'un candidat compatible est déterminé sans ambiguïté.	Actif
<code>services/accounting_service.py</code>	Orchestracteur de pré-comptabilité : contrôle les montants, le sens achat/vente, le tiers, les comptes HT/TVA/tiers, les devises, puis crée ou réutilise la pré-écriture ; crée aussi les mouvements d'un relevé.	Actif
<code>services/ligne_comptable_service.py</code>	Construit et synchronise les lignes débit/crédit des factures, banques et clôtures ; produit Grand Livre et Balance à partir des lignes validées.	Actif
<code>services/etat_comptable_service.py</code>	Contrôle l'équilibre et l'exploitabilité des lignes avant leur utilisation par les états.	Actif
<code>services/tva_comptable_service.py</code>	Classe les comptes TVA, calcule les synthèses mensuelles/annuelles, gère le crédit reporté, les régularisations et la validation TVA V2.	Actif
<code>services/cpc_service.py</code>	Classe les comptes 6/7 dans les rubriques CGNC, calcule les résultats et compare N à N-1 sans inventer de variation.	Actif
<code>services/bilan_service.py</code>	Classe les comptes de bilan, traite amortissements/provisions, expose les comptes non classés et contrôle l'égalité actif/passif.	Actif
<code>services/precloture_service.py</code>	Charge un snapshot du dossier et agrège les anomalies documents, écritures, banque, devises, TVA, clôture, CPC et Bilan ; calcule score et statut.	Actif
<code>services/cloture_service.py</code>	Valide les comptes exacts d'une régularisation, empêche les faux équilibres et génère/annule les lignes de clôture.	Actif
<code>services/rapport_service.py</code>	Agrège les indicateurs d'une entreprise pour une période et construit le schéma de rapport.	Actif

Fichier	Rôle et fonctionnement	État
<code>services/export_service.py</code>	Séréalise les registres en Excel, CSV ou PDF et génère les rapports PDF. Une fonction de format Topaze existe, mais l'API publique la bloque actuellement.	Actif

4.5.3 Services banque, devises et organisation

TABLE 4.7: Autres services métier

Fichier	Rôle et fonctionnement	État
<code>services/bank_account_service.py</code>	Normalise RIB/IBAN, cherche un compte bancaire compatible et l'affecte à un mouvement uniquement si l'identification est sûre.	Actif
<code>services/rapprochement_bancaire_service.py</code>	Calcule candidats et scores, gère allocations simples/partielles/groupées, confirmation, annulation et virements internes.	Actif
<code>services/exchange_rate_service.py</code>	Télécharge et parse les cours BAM, vérifie la date, met en cache, respecte l'unité de cotation et convertit séparément en MAD.	Actif
<code>services/exchange_difference_service.py</code>	Calcule gain/perte de change lors d'une allocation, y compris au prorata d'un paiement partiel, puis prépare les lignes si les comptes existent.	Actif
<code>services/available_company_service.py</code>	Construit des requêtes EXISTS pour ne proposer dans chaque module que les entreprises ayant des données pertinentes, sans N+1.	Actif
<code>services/alertes_service.py</code>	Liste les pré-écritures non saisies et détecte les entreprises en retard.	Actif
<code>services/tache_service.py</code>	Détermine le retard et recrée la prochaine occurrence lorsqu'une tâche récurrente est terminée.	Actif
<code>services/audit_service.py</code>	Enregistre une action scoppée avec auteur et détails avant/après ; seulement les actions instrumentées sont tracées.	Actif

4.6 Tâches, utilitaires et scripts

TABLE 4.8: Tâches et outils backend

Fichier	Rôle et fonctionnement	État
<code>backend/app/tasks/__init__.py</code>	Déclare le package Celery.	Support
<code>backend/app/tasks/document_processing.py</code>	Tâche principale <code>process_document</code> . Orchestre extraction, fallbacks, classement, préparation comptable, banque, statuts et retry des erreurs transitoires (maximum trois).	Actif
<code>backend/app/tasks/ai_enrichment.py</code>	Définit une tâche d'enrichissement différé. Aucun appel actuel à <code>enrichir_document_ia.delay</code> n'a été trouvé : ne pas la présenter comme une étape active.	Ancien

Fichier	Rôle et fonctionnement	État
backend/ocr_worker_standalone.py	Prototype de sous-processus PaddleOCR persistant. Le fichier s'arrête après le signal <code>ready</code> et l'architecture retenue utilise désormais le thread de <code>ocr_service.py</code> .	Ancien
backend/diag.py	Diagnostic ponctuel de l'import et de l'encodage de <code>models/user.py</code> ; utile lors d'un problème local, pas au runtime.	Support
backend/scripts/create_first_user.py	CLI d'amorçage du premier utilisateur avec mot de passe haché.	Support
backend/scripts/configurer_cabinet_seguribat.py	Configure les informations initiales du cabinet SEGURIBAT dans la base.	Support
backend/scripts/importer_plan_comptable_csv.py	Importe un plan comptable CSV en appliquant les normalisations du service dédié.	Support
backend/scripts/repair_accounting_entries.py	Répare/reconstruit des pré-écritures existantes avec les règles actuelles ; à exécuter consciemment sur une base sauvegardée.	Support
backend/scripts/train_categorie_classifier.py	Entraîne le classifieur local de catégories à partir d'un corpus contrôlé.	Support
backend/benchmarks/benchmark_extraction.py	Mesure durée, mémoire, résultats et diagnostics du pipeline sur des documents choisis.	Support

4.7 Alembic et évolution de la base

TABLE 4.9: Configuration et migrations Alembic dans l'ordre

Fichier	Rôle et fonctionnement	État
backend/alembic.ini	Configuration CLI et logs Alembic ; l'URL réelle est remplacée par <code>settings.DATABASE_URL</code> .	Support
backend/alembic/env.py	Charge tous les modèles et fournit <code>Base.metadata</code> à Alembic en mode online ou offline.	Actif
backend/alembic/script.py.mako	Gabarit utilisé pour créer une nouvelle révision.	Support
backend/alembic/README	Aide standard du dossier de migrations.	Support
ba0aefae0e94_initial_tables.py	Socle initial : cabinets, utilisateurs, entreprises, chronos, documents et pré-écritures.	Actif
f29924d2e6e1_make_document_classification_fields.py	Ajuste la nullabilité des champs de classification pour autoriser un traitement progressif.	Actif
18e58386fcfa_ajout_type_erreur_et_error_code_sur_.py	Ajoute type et code d'erreur structurés au document.	Actif
e8fd7f46f06e_ajout_saisie_topaze_sur_ecritures_.py	Ajoute le suivi de saisie sur les pré-écritures.	Actif
21b970ad6d7e_ajout_table_taches_rappels_et_taches.py	Première évolution liée aux tâches/rappels.	Actif

Fichier	Rôle et fonctionnement	État
1cd6f5a62909_ht_tva_optionnels_et_mouvements.py	Autorise HT/TVA optionnels et prépare le support des mouvements bancaires.	Actif
65a34d21d0e5_create_taches_table.py	Stabilise/crée la table des tâches utilisée actuellement.	Actif
a7c9e2b4d610_ajout_saisie_topaze_documents.py	Ajoute le suivi de saisie au niveau du document.	Actif
b91f2d7c4a80_create_mouvements_bancaires_table.py	Crée la table des mouvements issus des relevés.	Actif
c4f8a2d9e310_comptes_et_libelle_ecritures.py	Ajoute les comptes proposés et le libellé aux pré-écritures.	Actif
d7e3a1f6b420_plan_comptable_par_entreprise.py	Introduit le plan comptable propre à chaque dossier.	Actif
f1a6c9e4d230_rapprochement_bancaire.py	Ajoute le premier rapprochement entre mouvement et pré-écriture.	Actif
g3b8d2f5c640_moteur_devises_bam.py	Ajoute cours BAM et données de conversion séparées.	Actif
h4c9e6a7d510_lignes_comptables_grand_livre_balance.py	Ajoute les lignes débit/crédit, le Grand Livre et la Balance.	Actif
c8a1d2e3f470_banque_v2_allocations_comptes.py	Ajoute comptes bancaires, allocations multiples, paiements partiels et virements internes.	Actif
a9d4e7f2b681_devises_v2_ecarts_change.py	Étend les devises et les écarts de change au rapprochement.	Actif
b0e5f8a3c792_tva_v2_periodes_credits_regularisations.py	Ajoute configuration, périodes, crédits et régularisations TVA V2.	Actif
c1f6a9b4d803_ecritures_cloture_v1.py	Ajoute les régularisations de clôture et leur lien avec les lignes.	Actif

Chaîne complète des révisions :

```
ba0aefae0e94 -> f29924d2e6e1 -> 18e58386fcfa -> e8fd7f46f06e
-> 21b970ad6d7e -> 1cd6f5a62909 -> 65a34d21d0e5 -> a7c9e2b4d610
-> b91f2d7c4a80 -> c4f8a2d9e310 -> d7e3a1f6b420 -> f1a6c9e4d230
-> g3b8d2f5c640 -> h4c9e6a7d510 -> c8a1d2e3f470 -> a9d4e7f2b681
-> b0e5f8a3c792 -> c1f6a9b4d803
```

4.8 Tests backend

TABLE 4.10: Fichiers backend/tests/

Fichier	Rôle et fonctionnement	État
test_accounting_rules.py	Nature économique vers familles CGNC achat/vente/TVA et refus des natures inconnues.	Support
test_account_resolution_achat.py	Résolution stricte des comptes d'achat, fournisseurs, ambiguïtés, isolation et cohérence des fallbacks Groq/Ollama.	Support
test_audit_service.py	Portée cabinet et conservation avant/après dans l'audit.	Support
test_available_company_service.py	Sélecteurs d'entreprises par module, exercices, EXISTS, isolation et absence de N+1.	Support
test_bank_statement_service.py	Conservation des lignes bancaires répétées et signalement des lignes incomplètes.	Support
test_banque_v2_service.py	RIB/IBAN, paiements partiels/groupés, frais, acomptes et équilibre des lignes.	Support
test_bilan_service.py	Classement bilan, amortissements/provisions, résultat CPC et déséquilibre visible.	Support
test_cloture_service.py	Types de régularisation, comptes exacts, extourne, report et isolation multi-tenant.	Support
test_company_service.py	Identification entreprise/tiers, sens achat/vente, variations OCR et isolation cabinet.	Support
test_cpc_service.py	Rubriques CGNC et résultats exploitation, financier, courant, non courant et net.	Support
test_document_file_security.py	Cohérence extension/MIME/signature, limite de taille et confinement du chemin de stockage.	Support
test_document_retry_policy.py	Prouve que seules les erreurs transitoires sont réessayées.	Support
test_etats_v2_service.py	CPC/Bilan V2, N/N-1, clôture, comptes inconnus, équilibre et isolation.	Support
test_exchange_difference_service.py	Gains/pertes de change, paiements partiels, comptes manquants et portée multi-tenant.	Support
test_exchange_rate_service.py	Parser BAM, date, précision, unité 100, sens achat/vente et montant MAD bancaire réel.	Support
test_extraction_amount_integrity.py	Interdit à Groq et au mapping de reconstruire ou remplacer silencieusement HT/TVA.	Support
test_ligne_comptable_service.py	Schémas débit/crédit achat, vente, banque, équilibre et refus d'automatiser un paiement partiel ambigu.	Support
test_plan_comptable_service.py	Normalisation, cohérence famille, tiers et usage banque.	Support
test_precloture_service.py	Score et anomalies de tous les modules, ordre de priorité, exercice vide et isolation.	Support
test_rapprochement_bancaire_service.py	Sens attendu, montant, date, tiers et score d'un candidat.	Support
test_tva_comptable_service.py	Familles TVA, synthèse, crédit technique, reprises et écarts facture/lignes.	Support
test_tva_v2_service.py	TVA à payer/crédit, report, régularisations, configuration, double usage et isolation.	Support

Chapitre 5

Frontend : rôle de chaque fichier

5.1 Entrée, contexte et composants

TABLE 5.1: Entrée et composants React

Fichier	Rôle et fonctionnement	État
frontend/src/main.tsx	Monte le composant App dans le DOM sous StrictMode et charge les styles globaux.	Actif
frontend/src/App.tsx	Déclare les routes, charge les pages à la demande, protège les routes privées et applique le layout.	Actif
frontend/src/index.css	Styles globaux et directives Tailwind.	Actif
frontend/src/App.css	Styles hérités du squelette Vite ; rôle limité si les pages reposent sur Tailwind.	Support
frontend/src/context/AuthContext.tsx	État global de session : relit le JWT, charge l'utilisateur, connecte et déconnecte.	Actif
frontend/src/components/ProtectedRoute.tsx	Attend la résolution de session puis redirige vers <code>/login</code> si aucun utilisateur n'est chargé.	Actif
frontend/src/components/Layout.tsx	Coquille principale : navigation, entreprise active, recherche, notifications, tâches, identité et déconnexion.	Actif
frontend/src/components/ResizableSidebar.tsx	Sidebar redimensionnable, bornée et persistée dans <code>localStorage</code> .	Actif
frontend/src/components/EntriesTablePage.tsx	Tableau réutilisable des achats, ventes et pré-écritures : filtres, pagination, totaux, exports, validation, rejet et suivi de saisie.	Actif
frontend/src/components/AnomalyBadge.tsx	Badge compact qui rend visibles les anomalies et leur détail.	Actif

5.2 Client HTTP et modules API

TABLE 5.2: Fichiers frontend/src/api/

Fichier	Rôle et fonctionnement	État
api/client.ts	Instance Axios, URL du backend, ajout automatique du JWT, déconnexion sur 401 et téléchargement de blobs.	Actif
api/authApi.ts	Connexion et récupération de l'utilisateur courant.	Actif
api/usersApi.ts	Liste, création et désactivation d'utilisateurs.	Actif

Fichier	Rôle et fonctionnement	État
api/cabinetApi.ts	Cabinet courant, mise à jour et informations système.	Actif
api/entreprisesApi.ts	Liste complète, liste disponible par module et choix stable de l'entreprise sélectionnée.	Actif
api/documentsApi.ts	Import, liste, ouverture, retraitement, suppression, validation/rejet, édition bancaire et export d'un document.	Actif
api/documentDetailApi.ts	Charge la vue détaillée consolidée d'un document.	Actif
api/chronosApi.ts	Documents du chrono et années disponibles avec filtres.	Actif
api/accountingApi.ts	Pré-écritures, banque, rapprochements, comptes bancaires, registres et TVA ; c'est le client métier le plus large.	Actif
api/ledgerApi.ts	Grand Livre, Balance et reconstruction contrôlée des lignes.	Actif
api/cpcApi.ts	CPC d'une entreprise et d'un exercice.	Actif
api/bilanApi.ts	Bilan d'une entreprise et d'un exercice.	Actif
api/controlsApi.ts	Contrôles de pré-clôture et score du dossier.	Actif
api/clotureApi.ts	Liste, création, validation et génération des régularisations.	Actif
api/dashboardApi.ts	Indicateurs du tableau de bord pour une période optionnelle.	Actif
api/notificationsApi.ts	Notifications calculées et statistiques.	Actif
api/rappelsApi.ts	Alertes de saisie et retards.	Actif
api/tachesApi.ts	CRUD, filtres et achèvement des tâches.	Actif
api/rapportsApi.ts	Rapport JSON et téléchargement PDF.	Actif
api/searchApi.ts	Recherche transversale utilisée par l'en-tête.	Actif

5.3 Types TypeScript

Chaque fichier reflète un ou plusieurs schémas Pydantic. Si le backend change un champ, le type TypeScript et la page consommatrice doivent être vérifiés ensemble.

TABLE 5.3: Fichiers frontend/src/types/

Fichier	Rôle et fonctionnement	État
types/auth.ts	Requête de login, réponse JWT et utilisateur courant.	Actif
types/user.ts	Rôles, utilisateur administrable et payload de création.	Actif
types/cabinet.ts	Cabinet, mise à jour et informations système.	Actif
types/entreprise.ts	Dossier comptable et indicateur de création automatique.	Actif
types/document.ts	Résumé documentaire, statut et métadonnées.	Actif
types/documentDetail.ts	Détail documentaire et résumé de pré-écriture associé.	Actif
types/chrono.ts	Ligne affichée dans le chrono numérique.	Actif
types/ecriture.ts	Statuts, types, pré-écriture, filtres et payload de correction.	Actif
types/mouvementBancaire.ts	Mouvements, allocations, candidats, virements, comptes et filtres bancaires.	Actif
types/ledger.ts	Grand Livre, Balance et résultat de reconstruction.	Actif
types/registre.ts	Registres et vues TVA V1/V2.	Actif

Fichier	Rôle et fonctionnement	État
types/cpc.ts	Rubriques, détails, comparaison et contrôles CPC.	Actif
types/bilan.ts	Rubriques et contrôles du Bilan.	Actif
types/controls.ts	Niveaux, anomalies, modules et résultat de pré-clôture.	Actif
types/cloture.ts	Types et cycle d'une régularisation de clôture.	Actif
types/dashboard.ts	Compteurs et indicateurs synthétiques.	Actif
types/notification.ts	Notification, liste et répartitions statistiques.	Actif
types/rappel.ts	Pré-écritures non saisies et entreprises en retard.	Actif
types/tache.ts	Statut, priorité, récurrence et payloads d'une tâche.	Actif
types/rapport.ts	Structure de synthèse des rapports.	Actif
types/search.ts	Résultat de recherche unifié et route cible.	Actif

5.4 Pages React

TABLE 5.4: Fichiers frontend/src/pages/

Fichier	Rôle et fonctionnement	État
pages/LoginPage.tsx	Formulaire de connexion ; transmet les identifiants et remet le JWT au contexte.	Actif
pages/DashboardPage.tsx	Indicateurs, statuts et alertes synthétiques provenant du dashboard backend.	Actif
pages/UploadPage.tsx	Glisser-déposer/import, validation locale des formats, liste des documents et actions de suivi.	Actif
pages/DocumentDetailPage.tsx	Écran central de contrôle : fichier original, extraction, correction, pré-écriture, mouvements et décision humaine.	Actif
pages/ChronosPage.tsx	Recherche et classement par entreprise, période et catégorie avec sidebar redimensionnable.	Actif
pages/RegistersPage.tsx	Configure EntriesTablePage.tsx pour toutes les pré-écritures.	Actif
pages/AchatsPage.tsx	Même tableau réutilisable filtré sur le type achat.	Actif
pages/VentesPage.tsx	Même tableau réutilisable filtré sur le type vente.	Actif
pages/RelevsBancairesPage.tsx	Mouvements, filtres, comptes, candidats, allocations, rapprochement et virements internes.	Actif
pages/ComptesBancairesPage.tsx	Configuration des comptes bancaires du dossier et de leurs comptes comptables associés.	Actif
pages/RegistreComptablePage.tsx	Consultation des registres validés et téléchargement CSV/XLSX/PDF.	Actif
pages/GrandLivrePage.tsx	Consultation par compte, totaux et reconstruction contrôlée des lignes.	Actif
pages/BalancePage.tsx	Solde débiteur/créiteur par compte et contrôle de l'équilibre global.	Actif
pages/TvaMensuellePage.tsx	Synthèse TVA et recalcul des périodes TVA V2.	Actif
pages/CpcPage.tsx	CPC, comparaison N/N-1 et anomalies de classement.	Actif
pages/BilanPage.tsx	Actif/passif, rubriques, comptes non classés et contrôle d'équilibre.	Actif
pages/PreCloturePage.tsx	Score, statut et anomalies par module avant clôture.	Actif

Fichier	Rôle et fonctionnement	État
pages/CloturePage.tsx	Régularisations, validation et génération des lignes de clôture.	Actif
pages/RappelsPage.tsx	Alertes, pré-écritures non saisies, tâches et suivi opérationnel.	Actif
pages/NotificationsPage.tsx	Liste et statistiques des notifications calculées.	Actif
pages/RapportsPage.tsx	Choix entreprise/période, aperçu synthétique et PDF.	Actif
pages/AdminUsersPage.tsx	Gestion des utilisateurs : liste, création et désactivation.	Actif
pages/CnssPage.tsx	Ancien prototype avec données codées en dur et export local. Il n'est pas déclaré dans <code>App.tsx</code> .	Ancien
pages/HomePage.tsx	Ancienne page d'accueil non routée ; la racine redirige désormais vers le dashboard.	Ancien

5.5 Utilitaires, styles et ressources

TABLE 5.5: Autres fichiers frontend

Fichier	Rôle et fonctionnement	État
frontend/src/utils/tableExport.ts	Fonctions génériques d'export CSV et tableur côté navigateur pour les tableaux affichés.	Actif
frontend/src/assets/hero.png	Illustration de l'ancienne page d'accueil ; non nécessaire aux flux métier actuels.	Ancien
frontend/src/assets/react.svg	Logo du squelette React/Vite.	Ancien
frontend/src/assets/vite.svg	Logo du squelette Vite.	Ancien
frontend/public/favicon.svg	Icône de l'onglet navigateur.	Support
frontend/public/icons.svg	Sprite/ressource d'icônes publiques.	Support

Chapitre 6

Configuration, déploiement et documentation

TABLE 6.1: Fichiers d'infrastructure et de documentation

Fichier	Rôle et fonctionnement	État
requirements.txt	Dépendances Python directes nécessaires au backend.	Support
requirements-lock.txt	Versions Python verrouillées pour reproduire l'environnement.	Support
backend/current_env.txt	Photographie des paquets installés lors d'un diagnostic ; information locale, pas source primaire des dépendances.	Ancien
backend/Dockerfile	Image du backend avec dépendances et commande d'exécution.	Support
frontend/Dockerfile	Build multi-étapes du frontend puis service des fichiers statiques.	Support
docker-compose.yml	Développement : PostgreSQL, Redis, API, worker et volumes, avec ports locaux décalés.	Support
docker-compose.prod.yml	Variante de production avec configuration et politiques de redémarrage adaptées.	Support
frontend/nginx.conf	Sert la SPA, redirige les routes vers index.html et ajoute des headers de sécurité.	Support
frontend/package.json	Dépendances et commandes dev, typecheck, build, lint, preview.	Support
frontend/package-lock.json	Verrou exact de l'arbre npm.	Support
frontend/index.html	Coquille HTML contenant le point de montage root.	Support
frontend/vite.config.ts	Active le plugin React de Vite.	Support
frontend/tsconfig.json	Configuration TypeScript racine qui référence les variantes applicative et Node.	Support
frontend/tsconfig.app.json	Règles TypeScript de l'application : ES2023, DOM, JSX, bundler et contrôles de code inutilisé.	Support
frontend/tsconfig.node.json	Configuration TypeScript des fichiers exécutés par Node, notamment Vite.	Support
frontend/tailwind.config.js	Indique à Tailwind quels fichiers analyser pour générer les classes utilisées.	Support
frontend/postcss.config.js	Branche Tailwind et Autoprefixer dans PostCSS.	Support

Fichier	Rôle et fonctionnement	État
frontend/README.md	Documentation héritée du frontend ; à lire après la documentation racine.	Support
backend/pytest.ini	Ajoute backend au chemin Python et cible tests/ .	Support
README.md	Présentation générale et instructions principales du dépôt.	Support
README_FINAL_INSTALL.md	Procédure d'installation détaillée de l'environnement final.	Support
DEPLOYMENT.md	Consignes de déploiement et prérequis d'exploitation.	Support
STATUS_FINAL.txt	Instantané de statut du projet ; peut devenir obsolète par rapport au code.	Ancien
AGENTS.md	Règles de contribution et invariants comptables/multi-tenant du dépôt.	Support
rapport_comptaflow.tex	Source LaTeX du rapport académique principal.	Support
GUIDE_RAPPORT_LATEX.md	Instructions de compilation et de maintenance du rapport.	Support
fiche_revision_comptaflow.tex	Présente fiche, séparée du rapport afin de ne pas alourdir le document de soutenance.	Support

Chapitre 7

Carte de dépendances pour réviser rapidement

Si je modifie...	Je vérifie aussi...
Un champ de table	modèle SQLAlchemy, nouvelle migration, schémas Pydantic, service, route, type TypeScript, page et tests.
Une réponse API	schéma Out, fonction TypeScript correspondante, type TypeScript et rendu de la page.
Une règle de compte	<code>accounting_rules_service.py</code> , <code>plan_comptable_service.py</code> , <code>accounting_service.py</code> , génération des lignes et tests.
Le pipeline OCR/IA	prétraitement, Vision, OCR, Groq texte, regex, mapping, tâche Celery, statuts et tests d'intégrité.
Le rapprochement	modèles mouvement/allocation, service banque, lignes comptables, API, types, page Banque et tests Banque V2.
Un état comptable	uniquement lignes validées, service de calcul, schéma, endpoint, client TS, page et tests d'équilibre/isolation.
Un rôle utilisateur	enum backend, dépendance RBAC, routes sensibles, type frontend, affichage et tests 401/403.

7.1 Questions de soutenance et réponses courtes

1. **Pourquoi FastAPI ?** Pour des routes typées, la validation Pydantic, les dépendances d'authentification et une séparation claire avec React.
2. **Pourquoi Celery/Redis ?** Pour sortir OCR et IA du cycle HTTP, suivre l'état et réessayer seulement les erreurs transitoires.
3. **Pourquoi conserver les valeurs extraites ?** Elles sont la source d'audit ; les calculs contrôlent sans corriger silencieusement.
4. **Pourquoi l'IA ne choisit-elle pas directement les comptes ?** Les comptes exacts sont propres au plan de chaque entreprise et une ambiguïté doit rester visible.
5. **Qu'est-ce qui alimente les états ?** Les lignes débit/crédit effectivement validées, jamais une simple extraction brute.
6. **Comment l'isolation est-elle assurée ?** Le cabinet vient de l'utilisateur JWT et chaque requête métier ajoute les filtres cabinet et entreprise pertinents.
7. **Comment gérez-vous un paiement partiel ?** Une allocation porte le montant réellement affecté et laisse un solde restant sur la facture.
8. **Existe-t-il une intégration directe Topaze ?** Non. Des exports génériques existent ; l'endpoint

Topaze est volontairement non implémenté.

9. **Le traitement est-il strictement idempotent ?** Il résiste aux retraitements séquentiels, mais l'absence de certaines contraintes uniques laisse une limite en concurrence stricte.
10. **Quels fichiers sont centraux ?** Les points d'entrée sont `main.py` et `App.tsx`. Le pipeline principal est dans `document_processing.py`. La comptabilité repose surtout sur `accounting_service.py` et `ligne_comptable_service.py`. La banque utilise `rapprochement_bancaire_service.py` et la navigation React est structurée par `Layout.tsx`.

7.2 Ordre conseillé pour relire le code

1. `backend/app/main.py` et `frontend/src/App.tsx` pour voir les entrées.
2. `core/deps.py`, `models/base.py` et un modèle métier pour comprendre la sécurité.
3. `api/documents.py` puis `tasks/document_processing.py` pour le flux documentaire.
4. `accounting_service.py` puis `ligne_comptable_service.py` pour le cœur comptable.
5. `rapprochement_bancaire_service.py` pour Banque V2.
6. une chaîne frontend complète : page → API TS → route Python → service → modèle.
7. enfin les tests correspondants, qui donnent les exemples les plus précis des règles attendues.

Phrase à retenir : ComptaFlow automatise la préparation et le contrôle, mais conserve les données sources, utilise le plan comptable propre au dossier et exige une validation humaine dès qu'une décision comptable n'est pas suffisamment sûre.